Übungsblatt 6: Das Steuerwerk (Control Unit)

Mikrocomputertechnik 1

Zielsetzung

In diesem Übungsblatt überprüfen wir die Meistertabelle der Control Unit aus der Vorlesung. Sie leiten die theoretischen Steuersignale für einen unbekannten Befehl (Transferleistung) deduktiv her. Anschließend nutzen Sie den Ripes-Simulator, um Ihre Vorhersagen (Paper-&-Pencil-Methode) an den Steuerleitungen des Datenpfads zu verifizieren.

Aufgabe 1: Die Meistertabelle (Wiederholung)

Die Main Control Unit liest den 7-Bit-Opcode eines Maschinenbefehls und schaltet die Multiplexer und Enable-Pins des Datenpfads.

Das Signal `MemWrite`

Warum ist das Signal `MemWrite` das potenziell gefährlichste Signal im gesamten Prozessor? Was passiert, wenn dieses Signal bei einer Addition (`add`) fälschlicherweise auf 1 steht?

Don't Care bei `MemtoReg`

Das Steuersignal `MemtoReg` steht bei den Befehlen `sw` (Store) und `beq` (Branch) in unserer Wahrheitstabelle auf dem Wert X (Don't Care). Warum ist die Schalterstellung dieses Multiplexers in diesen Fällen völlig irrelevant für die CPU?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Aufgabe 2: Transferleistung — Signale für `addi`

In der Vorlesung haben wir die Signale für `add`, `lw`, `sw` und `beq` besprochen. Leiten Sie nun logisch die Signale für den Befehl `addi` (Add Immediate) her.

Tipp: `addi t0, t1, 5` liest eine Konstante aus dem Befehlswort (wie `lw`), speichert aber ein Rechenergebnis ins Register File (wie `add`).

Füllen Sie die folgende Zeile der Wahrheitstabelle aus (0, 1 oder X):

Instruktion	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
`addi`								

Ihre Lösung

(Tragen Sie Ihre Werte hier ein.)

Aufgabe 3: Verifikation in Ripes (Paper & Pencil)

Wir überprüfen Ihre Vorhersagen aus Aufgabe 2 an der Hardware. Öffnen Sie den Ripes-Simulator und wählen Sie den **Single-Cycle Processor**.

Schreiben Sie den folgenden Code:

```
.text
addi t0, x0, 10
```

Wechseln Sie in den Processor-Tab. Die Steuersignale des ersten Befehls liegen **kombinatorisch an, bevor** Sie Clock klicken (solange der PC auf `addi` zeigt). Nach einem Clock ist `addi` bereits abgeschlossen.

Suchen Sie die Control Unit. Beobachten Sie abgehende Steuerleitungen: in aktuellen Ripes-Versionen typisch **grün** $= 1$, **grau** $= 0$ (Tooltips lesen; Signalnamen können von `ALUSrc`/`RegWrite` abweichen).

Aufgabenstellung

Gleichen Sie die abgelesenen Werte in Ripes mit Ihrer Tabelle aus Aufgabe 2 ab.

1. Welchen Wert hat die Steuerleitung zum MUX vor dem zweiten ALU-Eingang (entspricht `ALUSrc`)?
2. Welchen Wert hat die Write-Enable-Leitung am Register File (entspricht `RegWrite`)?

Ihre Lösung

(Notieren Sie Ihre Beobachtungen hier.)

Aufgabe 4: Der MUX-Beobachter

Geben Sie den folgenden Code in Ripes ein:

```
.data
ziel: .word 0

.text
la s1, ziel
li t0, 5
add t1, t0, t0
sw t1, 0(s1)
beq t0, t1, Lbl
Lbl:
nop
```

Fokussieren Sie während des Durchklickens den Write-Back-MUX vor dem Register File (P&H: `MemtoReg`; in Ripes oft ein **3-Wege**-MUX „Reg wr src“).

Aufgabenstellung

Notieren Sie für `li`/`addi`, `add`, `sw` und `beq`, welchen **konkreten** Steuerwert Ripes anzeigt. Hinweis: Don’t Care (X) ist nur ein Tabellenkonzept — der Simulator zeigt immer 0 oder 1 (oder einen Mehrbit-Code). Bei `RegWrite = 0` ist der angezeigte Wert **egal** für die CPU — genau das bedeutet X.

Ihre Lösung

(Notieren Sie Ihre Beobachtungen hier.)